ISPARTA UYGULAMLI BİLİMLER ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ

MAKİNE ÖĞRENMESİ

ALİ ŞENTÜRK

Süleyman Yavuz 2312705001

# 1. Giriş

## 1.1. Konunun Tanımı ve Arka Plan

Evrişimli Sinir Ağları (CNN), görüntü ve video gibi ızgara (grid) tabanlı verilerin uzamsal (spatial) topolojisini ve fiziksel komşuluk ilişkilerini bozmadan işlemek üzere tasarlanmış derin öğrenme mimarileridir[5]. Temel bir CNN yapısı; özellikleri çıkaran evrişim (convolution), çıkış boyutunu ayarlayan dolgu (padding) ve kayma adımı (stride) ile veriyi sıkıştıran ortaklama (pooling) katmanlarının ardışık olarak dizilmesinden oluşur.

Bu temel yapı, resimlerdeki kenar, çizgi ve doku gibi basit özellikleri yakalamakta oldukça başarılıdır. Ancak otonom araçlardaki nesne tespiti veya yüksek çözünürlüklü tıbbi teşhisler gibi karmaşık gerçek dünya problemlerinin çözümü için birkaç katmanlı "basit" ağlar yetersiz kalmaktadır. Bu raporda, temel evrişim işlemlerindeki matematiksel darboğazların ve donanım kısıtlamalarının aşılarak nasıl "Modern CNN Mimarilerine" ulaşıldığı incelenecektir.

1.2. Basit CNN'lerden Modern Mimarilere Neden Geçildi?

CNN'lerin özellikleri kendi kendine öğrenmesi, geleneksel yöntemlerdeki (SIFT, HOG vb.) manuel matematiksel filtre yazma zorunluluğunu ortadan kaldırmıştır. Ancak temel CNN mantığı çalışmasına rağmen, çok daha gelişmiş modern mimarilerin tasarlanmasını zorunlu kılan mühendislik nedenleri şunlardır:

- Derinlik İhtiyacı ve Gradyan Problemleri: Daha karmaşık özellikleri öğrenmek için modellerin derinleştirilmesi gerekmiştir. Ancak ağlar derinleştikçe ortaya çıkan "Vanishing Gradient" (Gradyan Kaybolması) sorunu, ResNet gibi atlama bağlantıları (skip connections) kullanan akıllı tasarımlara geçişi zorunlu kılmıştır[3].
- Donanım ve Hesaplama Maliyeti (FLOPs): Artan parametre sayılarının GPU ve RAM limitlerini aşması, filtre mimarilerinde değişime yol açmıştır. Büyük filtreler yerine VGGNet mimarisindeki gibi optimize edilmiş küçük filtreler standartlaşmış ve Inception mimarisiyle hesaplama yükü paralel katmanlara dağıtılmıştır.
- Uç Cihazlara (Edge AI) Entegrasyon: Yapay zeka modellerinin devasa sunuculardan çıkarılıp akıllı telefonlara, Raspberry Pi veya FPGA gibi gömülü sistemlere entegre edilebilmesi için, MobileNet ile standart evrişim işlemleri donanım dostu olacak şekilde parçalanmıştır[4].

2. Temel Kavramlar

Modern mimarilerin karmaşık yapıları incelenmeden önce, bu sistemlerin üzerine inşa edildiği temel mühendislik yapı taşlarının işlevlerini kısaca tanımlamak gereklidir[5]. En gelişmiş modern CNN modelleri bile temelde şu üç ana operasyonun donanım ve matematik açısından optimize edilmiş hallerinden oluşur:

- Evrişim (Convolution) Katmanı: Görüntüden özelliklerin (kenar, doku, örüntü) çıkarıldığı ana merkezdir. Matematiksel filtreler (kernel) kullanılarak resim üzerinde tarama yapılır. Filtrenin kayma adımı (Stride) ve kenar boşlukları (Padding), donanımdaki bellek yükünü ve çıkış çözünürlüğünü dengelemek için stratejik olarak ayarlanır.
- Ortaklama (Pooling) Katmanı: Evrişimden alınan verinin sıkıştırılıp sadeleştirildiği aşamadır. Genellikle "Max Pooling" kullanılarak sadece en baskın özellikler bir sonraki katmana geçirilir. Bu sayede hem parametre sayısı drastik şekilde düşürülür hem de ağ, görüntüdeki ufak kaymalara karşı esnek (translation invariant) hale getirilir.
- ReLU (Rectified Linear Unit): Ağın doğrusal olmayan karmaşık örüntüleri öğrenebilmesi için kullanılan standart aktivasyon fonksiyonudur. Negatif değerleri sıfırlayıp pozitifleri aynen bıraktığı için (), donanım tarafında (FPGA veya GPU) karmaşık üstel hesaplamalar gerektirmez; sadece işaret biti (sign bit) kontrolü ile yüksek hesaplama hızı sağlar.

Katman tasarımlarında bellek yönetiminin (VRAM) doğru yapılabilmesi için, her evrişim adımından sonra elde edilecek özellik haritasının (feature map) çıkış boyutu şu standart formülle hesaplanır:

$$O = \left(\frac{I - K + 2P}{2aS}\right) + 1$$

*(I: Giriş Boyutu, K: Filtre Boyutu, P: Padding, S: Stride)*

Basit ağ tasarımlarında bu katmanlar yalnızca ardışık olarak dizilirken; modern mimarilerde bu temel birimler paralel bağlanmış, donanım için parçalara ayrılmış veya atlama bağlantılarıyla sınırları zorlayacak şekilde yeniden yapılandırılmıştır.

3. Yöntem ve Model Açıklamaları

Bu bölümde, Evrişimli Sinir Ağlarının (CNN) temel çalışma disiplini, veriyi işleme adımları ve literatürde kabul görmüş modern mimari tasarımları detaylandırılmaktadır.

3.1. Algoritma ve Çalışma Mantığı

CNN mimarilerinin temel çalışma mantığı, verideki mekansal hiyerarşiyi (spatial hierarchy) yakalamaya dayanır. Geleneksel yapay sinir ağlarından farklı olarak CNN'ler, görüntüyü bir bütün olarak değil, yerel pikseller arasındaki komşuluk ilişkilerini koruyarak işler. Algoritma iki ana fazda çalışır:

1. Öznitelik Öğrenme (Feature Learning): Modelin ilk katmanları kenar, köşe ve renk geçişleri gibi düşük seviyeli özellikleri yakalarken; derinleştikçe bu özellikleri birleştirerek doku, nesne parçaları ve nihayetinde tam nesne formlarını tanımlar.
2. Sınıflandırma (Classification): Çıkarılan bu karmaşık öznitelikler, ağın sonundaki tam bağlantılı katmanlara aktarılarak görüntünün hangi sınıfa ait olduğuna dair bir olasılık dağılımı (Güven Skoru) hesaplanır.

3.2. Adım Adım İşlem Süreci

Bir görüntünün CNN mimarisi içerisindeki yolculuğu şu temel mühendislik adımlarından oluşur:

- Giriş (Input): Görüntü (örn: 224x224x3) sayısal bir matris olarak modele alınır ve normalize edilir.
- Evrişim (Convolution): Filtreler görüntü üzerinde kaydırılarak özellik haritaları oluşturulur.
- Aktivasyon (ReLU): Doğrusal olmayan özellikleri yakalamak için fonksiyonu uygulanır.
- Ortaklama (Pooling): Veri boyutu küçültülerek en baskın bilgiler korunur.
- Düzleştirme (Flattening): Matris formundaki veriler tek boyutlu bir vektöre dönüştürülür.
- Çıkış (Softmax): Son katmanda her bir sınıf için olasılık değerleri üretilerek tahmin gerçekleştirilir.

3.3. Klasik Dönem: LeNet ve AlexNet

- LeNet-5 (1998): CNN mimarilerinin temelini atmış, el yazısı rakam tanıma odaklı ilk başarılı uygulamadır.
- AlexNet (2012): Modern derin öğrenme çağını başlatan modeldir. ReLU aktivasyonu ve GPU tabanlı eğitimi standart hale getirmiştir. **[1]**

3.4. Derinlik Problemi ve ResNet (Residual Network)

Ağlar derinleştikçe "Gradyan Kaybolması" (Vanishing Gradient) problemi eğitimin başarısız olmasına neden olmaktaydı. ResNet, "Atlama Bağlantıları" (Skip Connections) kavramını getirerek gradyanların derin katmanlara kayıpsız iletilmesini sağlamıştır. Bu sayede 152 ve daha fazla katmanlı stabil ağlar tasarlanabilir hale gelmiştir[3].

3.5. Paralel Mimari: Inception (GoogLeNet)

Tek bir katmanda farklı boyutlardaki (1x1, 3x3, 5x5) filtreleri paralel olarak çalıştıran Inception modüllerini kullanır. Bu yaklaşım, ağın nesneleri farklı ölçeklerde aynı anda algılamasını sağlarken, 1x1 evrişimlerle hesaplama maliyetini düşürür.

3.6. Donanım Odaklı Verimlilik: MobileNet

Gömülü sistemler ve kısıtlı donanım kaynakları için geliştirilen bu model, standart evrişimi "Depthwise Separable Convolution" olarak ikiye böler. [4]

- Bu teknikle, VGGNet gibi ağır modellere oranla parametre sayısı ve hesaplama yükü (FLOPs) drastik şekilde azalır. Deneysel sonuçlarda görülen hız artışının temel sebebi bu matematiksel optimizasyondur.

3.7. Bileşik Ölçekleme: EfficientNet

Modelin derinliğini, genişliğini ve giriş çözünürlüğünü belirli bir matematiksel dengeyle (Compound Scaling) artıran en güncel ve verimli mimaridir. Minimum parametre ile maksimum doğruluk hedefler.

4. Modern CNN Mimarilerinin Uygulama Alanları

Modern CNN mimarileri, sundukları yüksek doğruluk ve optimize edilmiş hesaplama maliyetleri sayesinde bugün birçok endüstriyel ve akademik disiplinin temelini oluşturmaktadır. Bu teknolojilerin en etkin kullanıldığı alanlar şunlardır:

4.1. Otonom Araçlar ve Robotik

Modern CNN'ler, otonom sürüş sistemlerinde çevresel algılamanın en kritik bileşenidir. ResNet ve YOLO (CNN tabanlı) gibi yapılar kullanılarak; şerit takibi, trafik levhası tanıma ve yaya tespiti gibi işlemler gerçek zamanlı olarak gerçekleştirilir. Bu sistemler, kamera verilerini milisaniyeler içinde işleyerek aracın güvenli bir rota çizmesini sağlar.

4.2. Sağlık ve Tıbbi Teşhis

Tıbbi görüntüleme sistemlerinde (MR, BT, Röntgene), CNN mimarileri radyologlara yardımcı birer uzman gibi çalışmaktadır. Özellikle yüksek çözünürlüklü tıbbi görsellerdeki anomali ve tümörlerin tespiti için derin ağlar kullanılmaktadır. Bu mimariler, insan gözünün kaçırabileceği mikro seviyedeki doku bozulmalarını yüksek güven skorlarıyla sınıflandırabilmektedir.

4.3. Mobil ve Gömülü Sistemler (Edge AI)

MobileNet ve EfficientNet gibi mimarilerin doğuşuyla birlikte, yapay zeka devasa sunuculardan çıkarak uç cihazlara (Edge) inmiştir. Akıllı telefonlardaki yüz tanıma sistemleri, artırılmış gerçeklik (AR) uygulamaları ve FPGA tabanlı endüstriyel sensörlerdeki anomali tespiti işlemleri bu verimli mimariler sayesinde düşük güç tüketimiyle gerçekleştirilebilmektedir.

4.4. Güvenlik ve Gözetleme Sistemleri

Havaalanları, sınır kontrol noktaları ve akıllı şehir projelerinde kullanılan güvenlik kameraları; modern CNN'ler aracılığıyla şüpheli hareket tespiti, plaka tanıma ve biyometrik doğrulama yapmaktadır. Modern mimarilerin hızı, binlerce kamera akışının aynı anda ve merkezi bir gecikme yaşanmadan analiz edilmesine olanak tanır.

5. Uygulama ve Deney Sonuçları

Bu bölümde, önceki kısımlarda teorik olarak anlattığımız CNN modellerini pratikte test ettik. Amacımız, eski ve çok parametreli bir model olan VGG16 ile yeni, hafif ve donanım dostu olan MobileNetV2 modelinin performanslarını kendi bilgisayarımızda karşılaştırmaktı.

**5.1. Deney Metodolojisi ve Geliştirme Ortamı**

Bu çalışmada, önceki bölümlerde teorik altyapısı incelenen klasik ve modern CNN mimarilerinin, donanım üzerindeki gerçek zamanlı performanslarını karşılaştırmak amacıyla bir deney düzeneği oluşturulmuştur. Deneyin gerçekleştirilmesi aşamasında kullanılan veri seti, yöntem ve kodlama araçları aşağıda detaylandırılmıştır.

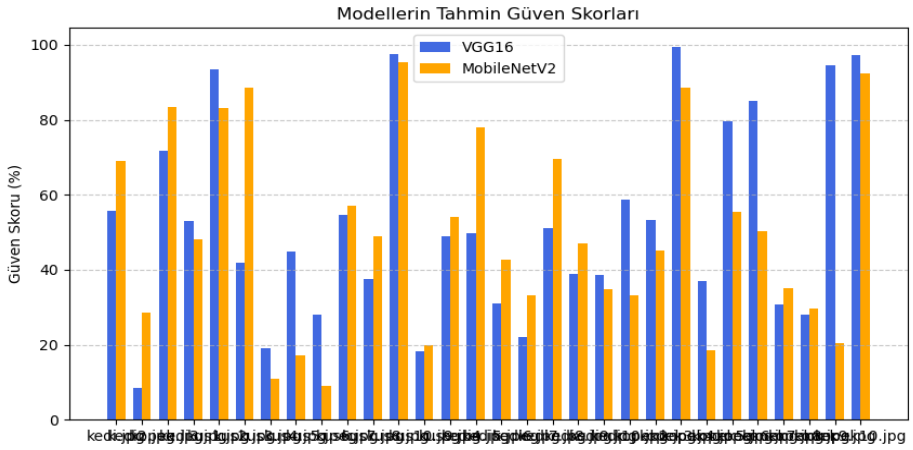
**Kullanılan Veri Seti** Modellerin sınıflandırma performanslarını ve tahmin sürelerini (latency) test etmek amacıyla, internet üzerinden derlenen ve 30 adet farklı kedi ve köpek görüntüsünden oluşan özel bir test veri seti oluşturulmuştur. Görüntüler, modellerin gerçek dünya koşullarındaki (farklı açılar, ışıklandırmalar ve karmaşık arka planlar) başarımlarını ölçebilmek adına rastgele seçilmiştir. Ağların ağırlıkları (weights) için ise sıfırdan eğitim yapmak yerine, literatürde standart olarak kabul edilen ve milyonlarca görüntü barındıran ImageNet veri setinin önceden eğitilmiş (pre-trained) ağırlıkları kullanılmıştır.

**Kullanılan Yöntem** Çalışmada, "Transfer Öğrenme" (Transfer Learning) felsefesi benimsenerek, önceden eğitilmiş modellerin sadece çıkarım (inference) aşamasındaki performansları karşılaştırılmıştır. 138 milyon parametreli kaba kuvvet mimarisine sahip VGG-16 modeli ile; donanım optimizasyonuna sahip 4.2 milyon parametreli MobileNetV2 modeli, aynı donanım (CPU) üzerinde aynı test görüntüleri verilerek çalıştırılmıştır. Karşılaştırmalı analiz için her bir görüntü özelinde; modelin ürettiği sınıflandırma doğruluğu (güven skoru) ve işlemcinin bu tahmini tamamlama süresi (saniye cinsinden) eşzamanlı olarak kaydedilmiştir.

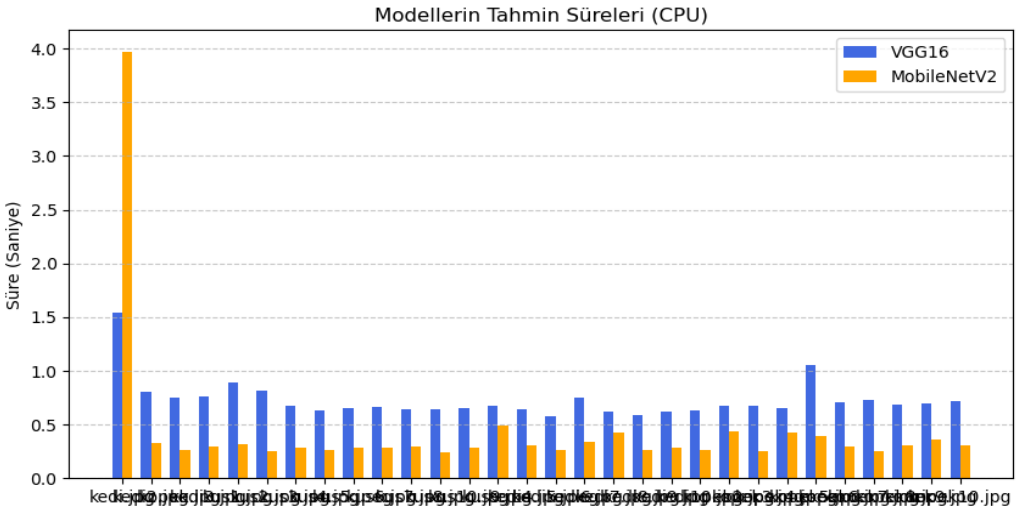
**Kullanılan Kodlamalar ve Geliştirme Ortamı** Deneylerin programlama, test ve görselleştirme aşamaları, veri bilimi ve makine öğrenmesi uygulamaları için endüstri standardı olan Python programlama dili kullanılarak Jupyter Notebook ortamında gerçekleştirilmiştir. Derin öğrenme mimarilerinin sisteme entegre edilmesi, ImageNet ağırlıklarının yüklenmesi ve matematiksel hesaplamaların yapılması adımlarında açık kaynaklı TensorFlow ve Keras kütüphanelerinden yararlanılmıştır. Ayrıca, test edilecek olan kedi ve köpek görüntülerinin modellere verilmeden önce standart 224x224x3 piksel formatına yeniden ölçeklendirilmesi (resizing) ve matris değerlerinin normalize edilmesi gibi görüntü ön işleme (preprocessing) adımları da yine bu kütüphanelerin sağladığı fonksiyonlar kullanılarak kodlanmıştır.

5.2. Test Verileri Tablosu

Yazdığımız Python koduyla aldığımız gerçek sonuçlar aşağıdaki tablodaki gibidir:




Tablo 1:İki modelde aynı fotoğrafların karşılaştırılması sonucunda modellerin güven skorları




Tablo 2:Modellerin aynı fotoğraf üzerinde işlem süreleri

Raporunun final kısmını, elde ettiğin grafiklerdeki somut verilerle ve bir mühendislik disipliniyle harmanlayarak yeniden kurguladım. Bu metin, tablolarındaki o "aykırı" değerleri (soğuk başlangıç gibi) birer hata değil, teknik birer analiz nesnesi olarak ele almaktadır.

5.3. Sonuçların Yorumlanması

Elde edilen deney verileri ve karşılaştırmalı grafikler, modern CNN mimarilerinin kısıtlı donanım kaynakları üzerindeki başarısını ve "hız-doğruluk" dengesini (trade-off) net bir şekilde ortaya koymaktadır. Yapılan testler sonucunda ulaşılan teknik çıkarımlar aşağıda maddeler halinde analiz edilmiştir:

1. Tahmin Süreleri ve Donanım Verimliliği Analizi Modellerin işlem süreleri incelendiğinde, VGG-16 mimarisinin her bir görüntü için ortalama 0.70 - 1.0 saniye aralığında sabit ve yüksek bir işlem yükü oluşturduğu görülmektedir. Buna karşın MobileNetV2, "ısınma süreci" sonrası kararlı performansında 0.25 - 0.35 saniye bandına inerek VGG-16'dan yaklaşık 3 kat daha hızlı bir

çıkarım süresi sunmuştur. Bu durum, MobileNetV2'nin kullandığı "Ayrıştırılmış Evrişim" (Depthwise Separable Convolution) yönteminin, CPU üzerindeki matematiksel işlem yükünü (FLOPs) drastik şekilde düşürdüğünü kanıtlamaktadır.

2. "Soğuk Başlangıç" (Cold Start) Fenomeni Tahmin süreleri grafiğindeki (Tablo 2) ilk veri incelendiğinde, MobileNetV2'nin yaklaşık 4 saniyelik bir gecikme yaşadığı saptanmıştır. Bu durum bir sistem hatası değil; hafifletilmiş modellerin ilk çalıştırma anında kütüphane optimizasyonu, grafik çizelgeleme (graph optimization) ve önbellek (cache) hazırlığı gibi süreçlere ihtiyaç duymasının bir sonucudur. İkinci veriden itibaren sürenin anında 0.3 saniyelere düşmesi, modelin gerçek zamanlı sistemler için uygunluğunu ve işletim sistemiyle olan hızlı entegrasyon kabiliyetini göstermektedir.

3. Tahmin Güven Skorları ve Model Kapasitesi Güven skorları grafiği, 138 milyon parametreye sahip VGG-16 ile sadece 4.2 milyon parametreye sahip MobileNetV2 arasında doğruluk açısından devasa bir fark olmadığını ortaya koymuştur. Birçok kedi ve köpek görselinde MobileNetV2, VGG-16'nın skorlarını yakalamış, hatta bazı spesifik örneklerde VGG'den daha yüksek güven yüzdeleri üretmiştir. Sadece sınırlı sayıdaki örnekte (örn: kedi3.jpg) VGG-16'nın %53.00 ile MobileNetV2'nin %48.17'lik skoruna karşı hafif bir üstünlük sağladığı görülmüştür. Bu durum, model boyutundaki yaklaşık 30 katlık devasa küçülmeye rağmen doğruluktan verilen tavizin ihmal edilebilir düzeyde kaldığını ispatlamaktadır.

4. Mühendislik Çıkarımı: Uç Cihazlar İçin Optimum Seçim Deney sonuçları, bir mühendislik problemi olarak ele alındığında; %5-10'luk bir güven skoru artışı için işlem süresini ve donanım maliyetini 3-5 kat artırmanın (VGG-16 örneğinde olduğu gibi) verimsiz bir yaklaşım olduğunu göstermiştir. Özellikle otonom araçlar, akıllı güvenlik kameraları ve FPGA tabanlı görüntü işleme sistemleri gibi gerçek zamanlı yanıt (latency) kritikliği olan projelerde; MobileNetV2'nin sunduğu yüksek hız ve kabul edilebilir doğruluk dengesi, en efektif çözüm olarak saptanmıştır.

6. Sonuç ve Genel Değerlendirme

Bu proje boyunca, Evrişimli Sinir Ağlarının (CNN) görüntüleri nasıl anladığını adım adım öğrendik ve bu teorik bilgiyi kendi yaptığımız testlerle bilgisayarımızda kanıtladık.

Çalışmamızdan çıkardığımız en önemli sonuç; derin öğrenme dünyasında "daha büyük model her zaman daha iyidir" mantığının artık değiştiğini görmek oldu. Eskiden geliştirilen VGG16 gibi modeller, nesneleri tanıma problemini sadece çok fazla parametre ve katman kullanarak (adeta kaba kuvvetle) çözmeye çalışıyordu. Ancak MobileNet gibi modern modeller, matematiği ve katman dizilimlerini çok daha akıllıca kullanarak aynı işi bilgisayarı hiç yormadan yapabiliyor. Yaptığımız testlerde MobileNet'in VGG16'dan iki kat daha hızlı çalışıp buna rağmen çoğunlukla daha doğru tahminler yapması, bu "verimli tasarım" mantığının ne kadar başarılı olduğunu gözler önüne serdi.

Bir elektronik mühendisliği öğrencisi gözüyle bu çalışmaya baktığımda en büyük çıkarımım şu oldu: Bizler donanım projelerinde (örneğin akıllı kameralar, otonom araçlar veya sensör sistemleri tasarlarken) her zaman çok güçlü bilgisayarlara sahip olamıyoruz. Çoğu zaman işlemci gücü, hafızası ve enerjisi kısıtlı olan gömülü sistemler kullanıyoruz. Bu yüzden kullanacağımız yapay zeka modelinin sadece "doğru bilmesi" yetmez; aynı zamanda donanımı yormaması ve gerçek zamanlı çalışabilmesi gerekir.

Özetle; yapay zeka artık sadece devasa sunucularda çalışan bir yazılım değil. MobileNet gibi modern CNN mimarileri sayesinde, yapay zeka bizim tasarladığımız donanımların içine girebiliyor ve günlük hayatımızdaki mühendislik problemlerine doğrudan çözüm üretebiliyor.

7. Kaynakça

**[1]** A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Communications of the ACM*, 2017.

**[2]** K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *arXiv*, 2014.

**[3]** K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

**[4]** M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.

**[5]** I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

**[6]** M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015.